

ESCALA PARA EL PROYECTO RAG CONTRA LA MÁQUINA (/PROJECTS/RAG-AGAINST-TI MACHINE)

Debes evaluar a 1 estudiante de este equipo.



Repositorio Git

`git@vogsphere-v2.42madrid.com:vogsphere/intra-uuid-52974867-52c`

Introducción

- Manténgase educado, respetuoso y constructivo durante la evaluación.
- Este proyecto evalúa la recuperación de información, la segmentación de texto, la generación de respuestas y el diseño general del sistema RAG.
- Los scripts de examen de referencia se proporcionan en ``exams/scripts/``:
``exam_retrieval.sh`, `exam_answer.sh`, `exam_edge_cases.sh``
- Consulte ``exams/exam_scripts_instructions.md`` para obtener instrucciones detalladas para el cc
- La corrección está diseñada para que el proceso de recuperación (~8 min) se ejecute en segundo plano mientras usted inspecciona el código y hace preguntas.
- Cada pregunta obligatoria es de aprobado/suspenso. Toda la parte obligatoria debe validarse (todas las preguntas deben aprobarse) antes de que se califique la bonificación.

Pautas

- Calificar únicamente el trabajo presente en el repositorio Git del estudiante.
- Configurar un espacio de trabajo de evaluación: clonar el repositorio del estudiante en un subdirectorio llamado ``student/``, y colocar el binario ``moulinette`` proporcionado y los scripts ``exams/`` junto a él.
- Ejecutar ``cd student && uv sync`` para instalar las dependencias del estudiante. Los archivos ``pyproject.toml`` y ``uv.lock`` del estudiante se encuentran en la raíz de su repositorio, por lo que se espera que ``uv sync`` funcione desde allí.
- La solución del estudiante NUNCA debe importar paquetes de moulinette. Verificar (coincidir solo con importaciones, no con la palabra en comentarios/cadenas de documento ``grep -rE "[[:space:]]*(import|from)[[:space:]]+moulinette" student/ --include="*.py"``
- El programa no debe fallar inesperadamente durante la evaluación.
- No se permite ninguna modificación del código del estudiante durante la corrección.
- Los conjuntos de datos se proporcionan con esta escala. Los estudiantes trabajan únicamente con conjuntos de datos públicos; Para la calificación, descomprima los conjuntos de datos privados exc corrector:
``unzip data/datasets/datasets_private.zip -d data/datasets/``.
- No se espera que estos conjuntos de datos privados estén en el repositorio del estudiante.
- Use indicadores si se detecta trampa o plagio.

Archivos adjuntos

 datasets_private.zip (https://cdn.intra.42.fr/document/document/54679/datasets_private.zip)

 exámenes.zip (https://cdn.intra.42.fr/document/document/54680/exams.zip)

 asunto.pdf (https://cdn.intra.42.fr/pdf/pdf/220267/en.subject.pdf)

 datasets_public.zip (https://cdn.intra.42.fr/document/document/54681/datasets_public.zip)

 LÉAME.md (https://cdn.intra.42.fr/document/document/54682/README.md)

 vllm-0.10.1.zip (https://cdn.intra.42.fr/document/document/54683/vllm-0.10.1.zip)

 moulina.zip (https://cdn.intra.42.fr/document/document/54684/moulinette.zip)

Parte obligatoria

Preliminares y preparación

Verifique los siguientes requisitos:

- El repositorio Git está disponible y contiene solo los archivos solicitados.
- Se utiliza Python 3.10+
- uv se utiliza como gestor de proyectos y paquetes.
- Los archivos pyproject.toml y uv.lock están presentes.
- El directorio src/ existe con el módulo principal.
- En la raíz del repositorio hay un archivo Makefile.
- El archivo README.md está presente.
- La sincronización UV se completa sin errores (ejecutar desde la raíz del repositorio del estur cd student && uv sync en el espacio de trabajo de evaluación).
- Paso exclusivo del corrector: descomprima los conjuntos de datos privados proporcionados unzip data/datasets/datasets_private.zip -d data/datasets/ (no se espera que el estudiante e de datos privados).

¿Se cumplen todos los requisitos de configuración?

✓ Sí

✗ No

Proceso de lanzamiento (Antecedentes)

Inicie el proceso de recuperación automatizada en una terminal en segundo plano. Esto ejecuta la indexación, la búsqueda y la evaluación (aproximadamente 8 minutos en total).

En una terminal aparte, ejecute:

```
./exams/scripts/exam_retrieval.sh \
--student-path ./student \
--moulinette-path ./moulinette-ubuntu
```

Continúe con la inspección de código (Q3-Q6) mientras se ejecuta. Conserve la salida de esta ejecución (registro de terminal y summary.log): las preguntas Q7 a Q10 obtienen sus resultados de este registro (documentación/código Recall@5, tiempo de indexación y rendimiento). No cierre la terminal antes de Q10.

¿Se puso en marcha el oleoducto con éxito?

✓ Sí

✗ No

Calidad del código y modelos Pydantic

Revisa la calidad del código y la implementación del modelo de datos. Verifica TODOS los sigui

Calidad del código:

- Los modelos de datos utilizan pydantic para la validación y la seguridad de tipos (las clases orquestación, como el indexador/recuperador/canalización, no necesitan usar pydantic).
- El código se ajusta a los estándares de codificación de flake8 y pasa mypy (ejecutar make 1 Instrucciones Comunes definen como flake8 + mypy).
- El archivo Makefile expone las reglas necesarias (instalar, ejecutar, depurar, limpiar, analizar
- Las excepciones se manejan de forma adecuada con bloques try-except.
- Los recursos se gestionan adecuadamente (gestores de contexto o close() explícito).
- Las barras de progreso se implementan para operaciones de larga duración (tqdm).

Modelos Pydantic (los 8 deben estar implementados correctamente):

- MinimalSource (ruta_archivo, índice_primer_carácter, índice_último_carácter)
- Pregunta sin respuesta y pregunta respondida
- Conjunto de datos Rag
- Resultados de búsqueda mínimos y respuesta mínima
- Resultados de búsqueda de estudiantes y Resultados de búsqueda de estudiantes y respue

Formato de salida:

- Las operaciones de búsqueda generan un JSON válido utilizando el modelo StudentSearchF
- Las operaciones de respuesta generan JSON válido utilizando el modelo StudentSearchRes
- Cada fuente incluye la ruta de archivo correcta, el índice del primer carácter y el índice del ú

¿El código cumple con todos los estándares de calidad y todos los modelos están implementados

✓ Sí

✗ No

Estrategias de segmentación

Verificar las estrategias de segmentación:

- Se implementan al menos dos estrategias de segmentación (por ejemplo, código Python + `max_chunk_size`).
- El tamaño de fragmento predeterminado es de 2000 caracteres y se puede configurar media
- El estudiante puede explicar el impacto del tamaño de los fragmentos en el rendimiento de la búsqueda. "¿Qué sucede si los fragmentos son demasiado pequeños? ¿O demasiado grandes?"

Pídele al estudiante que muestre el código de ambas estrategias.

¿Se han implementado correctamente las estrategias de segmentación?



Sí



No

Sistema de recuperación

Verificar la implementación de la recuperación:

- Se implementa al menos un método (TF-IDF o BM25).
- El sistema recupera y clasifica la información relevante.
- El estudiante puede explicar cómo funciona el método que ha elegido.

Demostración en vivo: pídale al estudiante que ejecute una consulta de búsqueda y explique los resultados. `uv run python -m src search "¿Cómo configurar el servidor OpenAI?" --k 10`

Is the retrieval system properly implemented?



Yes



No

Answer Generation

Verify answer generation:

- Qwen/Qwen3-0.6B is used as the DEFAULT model
- Run: `uv run python -m src answer "How to configure OpenAI server?" --k 10`
- System passes retrieved context to LLM within token limits
- Answers are generated based on retrieved code/documentation

Note: The student may support additional local models beyond the default. Check the README for documentation of supported models. During evaluation, always test with the default model (Qwen3-0.6B).

Does answer generation work correctly with the default model?



Yes



No

Retrieval Performance: Docs

Check the `exam_retrieval.sh` results.

The script evaluates Recall@5 on the private docs dataset (100 questions). Look for the "Docs Recall@5" line in the output.

Pass criterion: Docs Recall@5 >= 80% (the threshold stated in the subject).

Does the docs dataset meet the 80% Recall@5 threshold?



Yes



No

Retrieval Performance: Code

Check the `exam_retrieval.sh` results.

The script evaluates Recall@5 on the private code dataset (100 questions). Look for the "Code Recall@5" line in the output.

Pass criterion: Code Recall@5 >= 50% (the threshold stated in the subject).

Does the code dataset meet the 50% Recall@5 threshold?



Yes



No

Retrieval Performance: Indexing Time

Check the `exam_retrieval.sh` results.

Test 1/4 of the script times the full indexing run against the 300s (5 min) limit stated in the subject. Look for the "Indexing" line in the output (or the "INDEX:" line in summary.log).

Pass criterion: indexing completed in <= 300s (reported PASSED / "INDEX: OK", not EXCEEDED or CRASHED).

Did indexing complete within the 5-minute limit?

☒ Yes

☐ No

Retrieval Performance: Throughput

Check the exam_retrieval.sh results.

Test 2/4 of the script times search over the two 100-question datasets (200 questions total) against the 90s limit stated in the subject. Look for the "Warm Retrieval Throughput" line (or the "THROUGHPUT:" line in summary.log).

Pass criterion: 200 questions retrieved in <= 90s (reported PASSED / "THROUGHPUT: OK", not EXCEEDED or CRASHED).

Did retrieval stay within the 90s throughput limit?

☒ Yes

☐ No

Answer Quality

Run the answer exam script to evaluate answer quality:

```
./exams/scripts/exam_answer.sh \
--student-path ./student \
--moulinette-path ./moulinette-ubuntu
```

The script lists questions where sources were correctly retrieved, then you pick 3 questions. For each, the student's answer is displayed.

The mandatory Qwen/Qwen3-0.6B model has known reasoning limits, so grading prioritizes retrieval quality and grounding over perfect final phrasing. A satisfactory answer should:

- be coherent and understandable,
- be mostly grounded in the retrieved sources (no major hallucination),
- address the question asked. Minor incompleteness due to base-model limitations is acceptable

Pass if at least 2 out of 3 answers are satisfactory.

Does the student pass the answer evaluation (2/3 answers ok)?

☒ Yes

☐ No

Student Understanding

Test the student's understanding of RAG concepts and their implementation. Ask these 5 questions (in order):

1. "What is Retrieval-Augmented Generation (RAG) and why is it useful?"
2. "Walk me through your complete RAG pipeline, from raw documents to a generated answer."
3. "What is the difference between TF-IDF and BM25 as retrieval methods?"
4. "What implementation choices did you make and why? What trade-offs did you consider?"
5. "If you had more time, what would you improve in your system?"

Pass requires 4 out of 5 correct/satisfactory answers. A satisfactory answer demonstrates genuine understanding, not just memorized definitions.

Does the student demonstrate good understanding (4/5)?

☒ Yes

☐ No

README & Documentation

Verify README.md contains all required sections. Checklist:

1. [] System architecture (RAG pipeline components and interactions)
2. [] Chunking strategy explanation
3. [] Retrieval method details (algorithm and ranking mechanism)

- 4. ☐ Performance analysis (recall@k scores and system performance)
- 5. ☐ Design decisions and trade-offs
- 6. ☐ Example usage with clear instructions

If the student implemented bonus features, the README should also document those experiments and their results.

Pass if at least 5 out of 6 items are present and well-documented.

Is the README comprehensive and well-documented?

☒ Yes

☐ No

System Reliability

Run the edge case exam script:

`./exams/scripts/exam_edge_cases.sh --student-path ./student`

This tests 4 degenerate inputs:

- 1. Empty query: `search "" --k 10`
- 2. Gibberish query: `search "asdfghjkl" --k 10`
- 3. `k=0`: `answer "What is vLLM?" --k 0`
- 4. Bad dataset path: `search_dataset --dataset_path /nonexistent.json`

Pass criterion: ALL 4 tests complete without Python tracebacks. The program may print error messages or return empty results: that is acceptable. Unhandled exceptions with tracebacks are NOT acceptable.

Does the system handle all edge cases without crashing?

☒ Yes

☐ No

Bonus Part

Bonus: Advanced RAG

Graded only once the entire mandatory part is validated.

There are exactly five bonuses, each worth one point (one star). They all run on a CPU-only campus machine. Award one star per bonus that is implemented AND demonstrated working (not merely described in the README):

- 1. Semantic embeddings used for retrieval (CPU model, e.g. all-MiniLM-L6-v2)
- 2. Hybrid retrieval combining the lexical and semantic rankings
- 3. Incremental indexing (re-index only changed files)
- 4. Caching of the index and/or query results
- 5. Local HTTP API to query the index and answer questions

Calificación por estrellas = número de bonificaciones activas (0 -> 1 estrella, 5 -> 5 estrellas). Pídale al estudiante que demuestre cada bonificación obtenida.



Califícalo del 0 (fallido) al 5 (excelente).

Calificaciones

No olvides comprobar la bandera correspondiente a la defensa.

☒ De acuerdo

☐ Proyecto excepcional

☐ Trabajo vacío

☐ Trabajo incompleto

☐ Compilación no válida

☐ Norma

☐ Engañar

☐ Situación actual

☐ Función prohibida

☐ No puedo dar soporte ni explicar

Conclusión

Deja un comentario sobre esta evaluación (máximo 2048 caracteres).



Finalizar la evaluación

Condiciones generales de uso de la API
(<https://profile.intra.42.fr/legal/terms/33>)

Declaración sobre el uso de cookies
(<https://profile.intra.42.fr/legal/terms/2>)

Política de privacidad
(<https://profile.intra.42.fr/legal/terms/5>)

Condiciones generales de uso del sitio web
(<https://profile.intra.42.fr/legal/terms/6>)

Reglame
(<https://profile.intra.42>)